

DESIGN OF A SAFE SEMANTIC PLANNER IN A FINITE CARTESIAN STATE SPACE

PCCST503 – Machine Learning
Assignment 1

Submitted by:

Manasa M

TCR24CS044

B.Tech Computer Science and Engineering
S5 CSE

Government Engineering College, Thrissur

ABSTRACT

This project implements a Safe Semantic Planner for a finite Cartesian state space. States are represented using Cartesian coordinates and directed transitions contain cost, safety, reliability and availability information.

The planner finds a path from an initial state to a goal state while avoiding predefined bad states. Euclidean distance is used as a heuristic and safety measure. The planner also supports dynamic changes such as transition failure, goal updates and addition of new transitions.

Six test cases are used to verify reachability, bad-state avoidance, safety margin and dynamic replanning. The performance is evaluated using path cost, safety distance, explored states and planning time.

INTRODUCTION

Planning is the process of finding a sequence of actions that moves an agent from an initial state to a desired goal state. In this project, the environment is represented as a finite graph where states are nodes and directed transitions are edges.

Each state has a Cartesian representation. Each transition has cost, safety, reliability and availability properties. The planner must reach the goal without entering bad states while considering cost and safety.

PROBLEM DEFINITION

Given a finite set of Cartesian states, an initial state, a goal state, bad states and directed transitions, the objective is to find a valid and safe path from the initial state to the goal.

OBJECTIVES

- Reach the goal successfully.
- Avoid all bad states.
- Minimize path cost.
- Maintain a suitable safety distance.
- Consider transition reliability and availability.
- Replan when the environment change

SYSTEM DESIGN

The planner consists of the following modules:

1. State Module – stores state IDs and Cartesian coordinates.
2. Transition Module – stores directed transitions and their properties.
3. Graph Module – represents connections between states.
4. Safety Module – calculates distance from bad states.
5. Heuristic Module – estimates distance to the goal.
6. Planner Module – searches for a valid path.
7. Replanning Module – handles changes in the environment.
8. Evaluation Module – calculates performance measures.

STATE AND TRANSITION REPRESENTATION

A state contains a unique identifier and Cartesian coordinates.

Example:

State("A", (1, 2))

A transition connects two states and contains:

- Source
- Target
- Cost
- Safety
- Reliability
- Availability

Only available transitions leading to valid states are considered during planning.

PLANNING ALGORITHM

The planner performs the following steps:

1. Initialize states and transitions.
2. Define the initial and goal states.
3. Mark unsafe states as bad states.
4. Construct the transition graph.
5. Calculate heuristic values.
7. Ignore bad states and unavailable transitions.
8. Select the most promising state using the priority queue.
9. Continue until the goal is reached.
10. Reconstruct the final path.
11. Calculate cost, safety and reliability.

DYNAMIC REPLANNING

The planner can respond to changes such as:

A transition becoming unavailable.

Goal state changing.

A new transition being added.

After a change, the planner generates an updated valid path.

HEURISTIC FUNCTION

The heuristic function helps the planner find the goal faster.

It estimates how close the current state is to the goal state using the distance between their Cartesian coordinates.

A state that is closer to the goal gets a lower heuristic value. Therefore, the planner gives more priority to states that are likely to reach the goal quickly.

SAFETY COMPUTATION

Safety computation checks how far a state is from the nearest bad state.

If a state is very close to a bad state, it is considered less safe. The planner avoids bad states completely and prefers paths that maintain a better distance from them.

The safety value is also used while comparing different possible paths. Thus, the planner tries to find a path that is both efficient and safe.

IMPLEMENTATION

Software Requirements

- *Python 3.x
- *NumPy
- *Pandas
- *Matplotlib

Project Structure

SafeSemanticPlanner/

```
|  
|—— main.py  
|—— requirements.txt  
|—— README.md
```

Execution

Run the main program using:

```
python main.py
```

The program executes the defined test cases and displays the planning results and graphical output.

Output

The program reports information such as:

Success/failure

State path

Total cost

Minimum safety distance

Reliability

Explored states

Planning time

Graphs are generated to visualize the state space and selected path.

PROGRAM RESULT

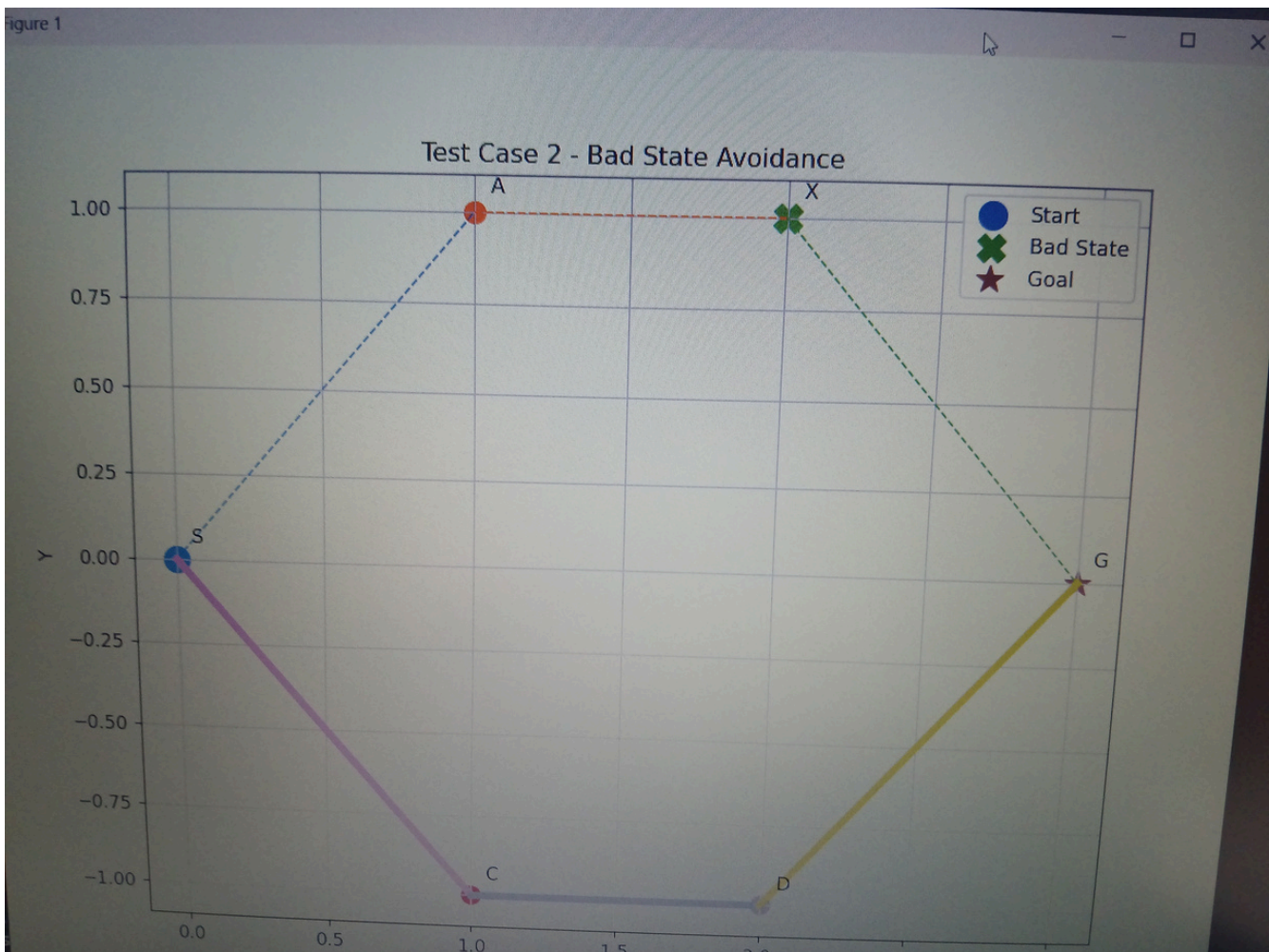
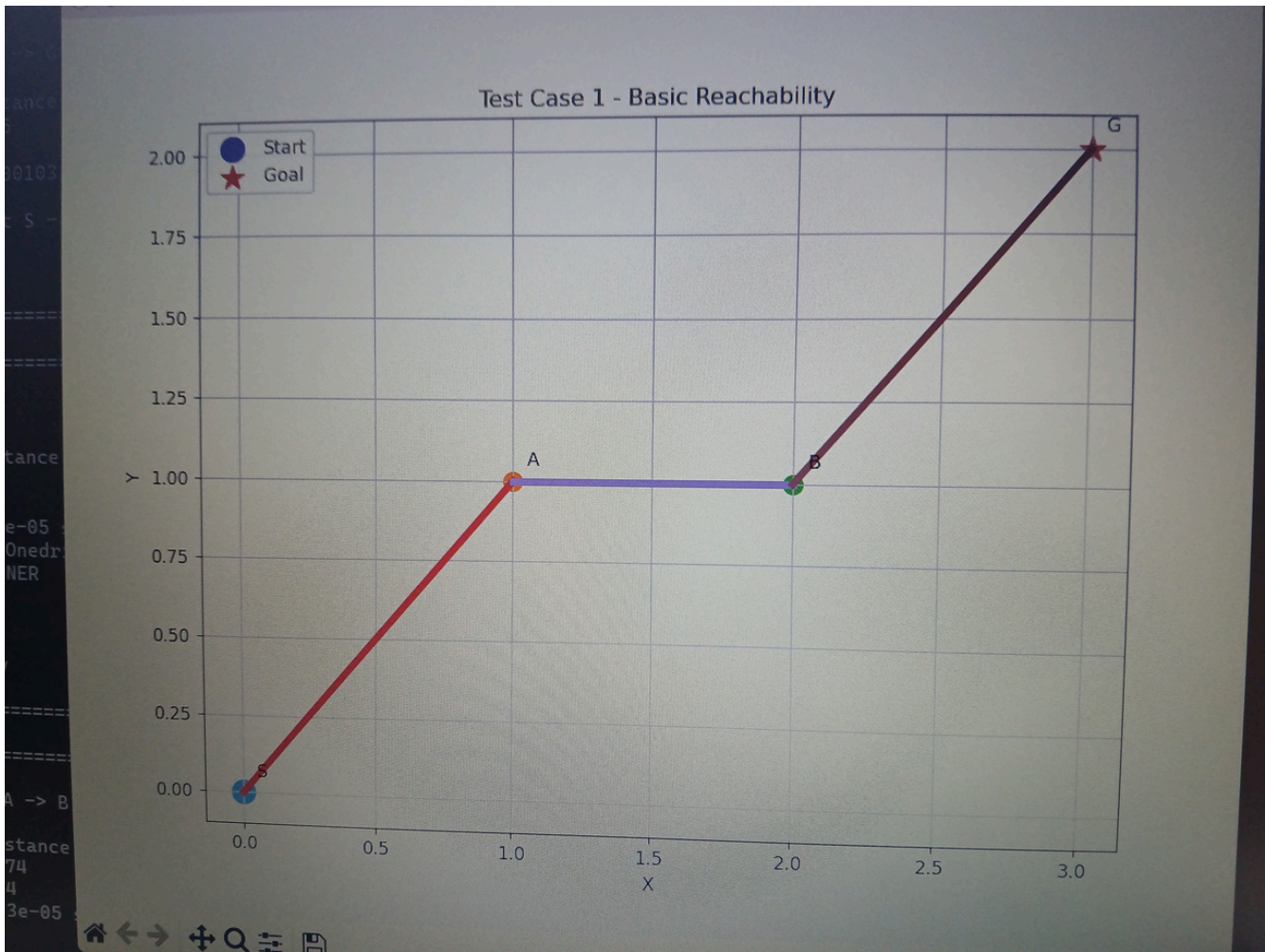
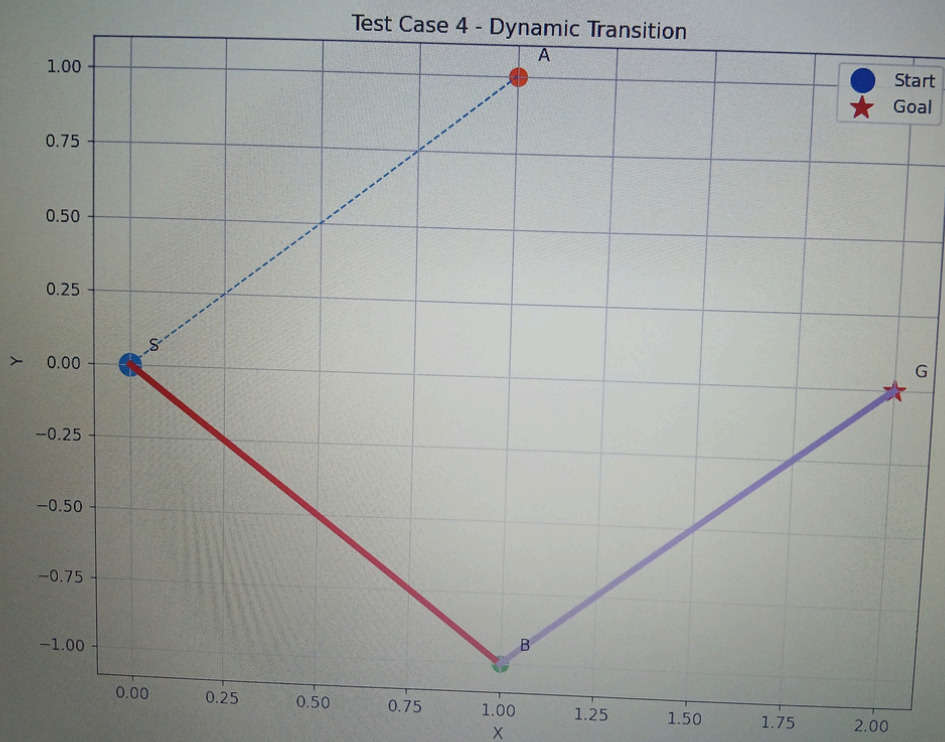
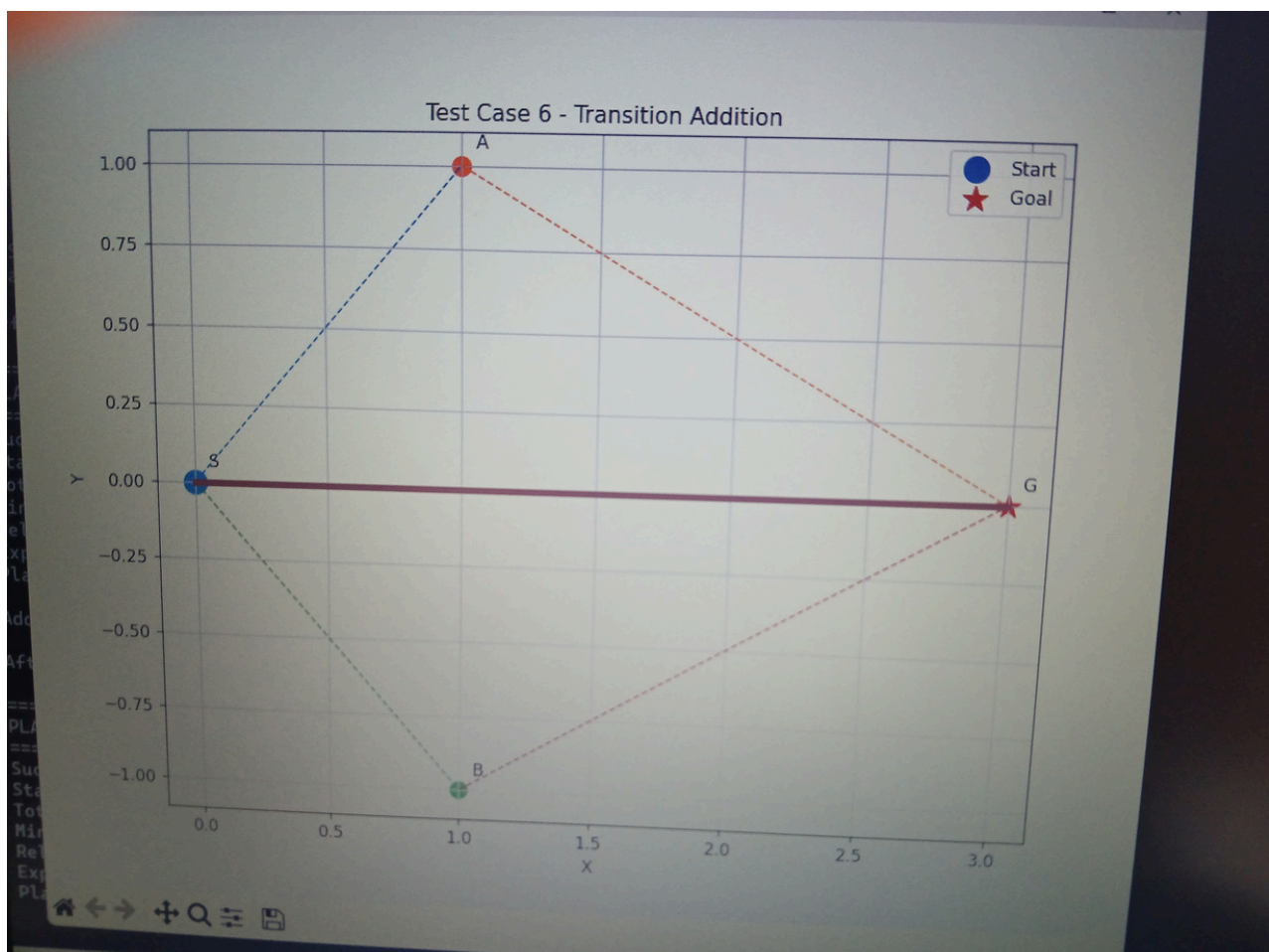
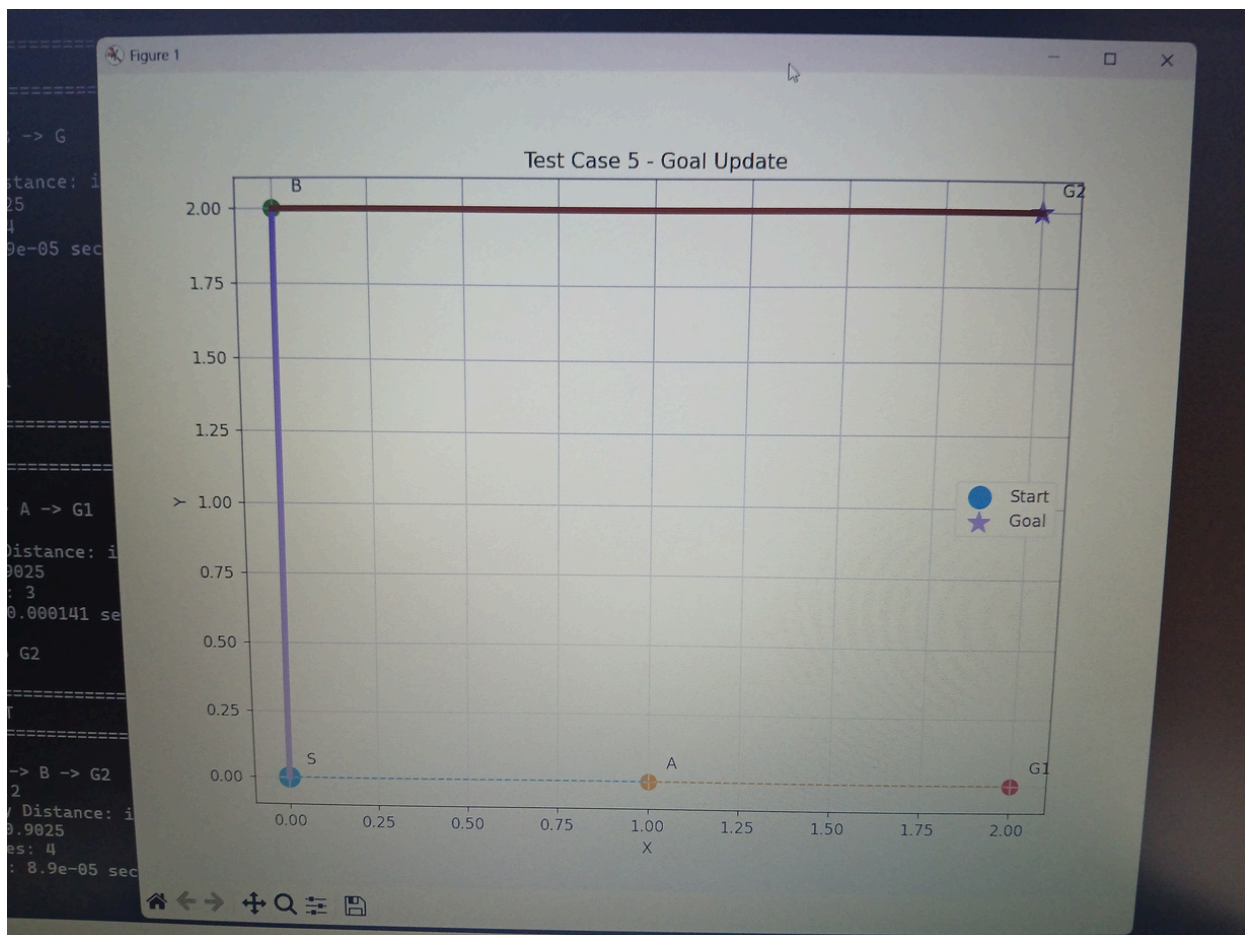


Figure 1



Figure 1





TEST CASE 1
Basic Reachability

=====

PLANNING RESULT

=====

Success: True
State Path: S -> A -> B -> G
Total Cost: 3.3
Minimum Safety Distance: inf
Reliability: 0.8574
Explored States: 4
Planning Time: 8.3e-05 seconds

TEST CASE 2
Bad State Avoidance

=====

PLANNING RESULT

=====

Success: True
State Path: S -> C -> D -> G
Total Cost: 14.5716
Minimum Safety Distance: 1.4142
Reliability: 0.8574
Explored States: 5
Planning Time: 8.4e-05 seconds

TEST CASE 3
Safety Margin

=====

PLANNING RESULT

=====

Success: True
State Path:

TEST CASE 3
Safety Margin

=====

PLANNING RESULT

=====

Success: True
State Path: S -> C -> D -> G
Total Cost: 25.2737
Minimum Safety Distance: 1.5811
Reliability: 0.8574
Explored States: 5
Planning Time: 0.00176 seconds

TEST CASE 4
Dynamic Transition

Initial planning:

=====

PLANNING RESULT

=====

Success: True
State Path: S -> A -> G
Total Cost: 2.2
Minimum Safety Distance: inf
Reliability: 0.9025
Explored States: 3
Planning Time: 0.000201 seconds

Transition A -> G becomes unavailable.
Replanning:

=====

PLANNING RESULT

=====

Success:

Minimum Safety Distance: inf
Reliability: 0.9025
Explored States: 3
Planning Time: 0.000201 seconds

Transition A -> G becomes unavailable.

Replanning:

=====

PLANNING RESULT

=====

Success: True
State Path: S -> B -> G
Total Cost: 4.2
Minimum Safety Distance: inf
Reliability: 0.9025
Explored States: 4
Planning Time: 8.9e-05 seconds

TEST CASE 5
Goal Update

Initial goal = G1

=====

PLANNING RESULT

=====

Success: True
State Path: S -> A -> G1
Total Cost: 2.2
Minimum Safety Distance: inf
Reliability: 0.9025
Explored States: 3
Planning Time: 0.000141 seconds
Goal changed to G2

=====

PLANNING RESULT

=====

Windows PowerShell

State Path: S -> A -> G1
Total Cost: 2.2
Minimum Safety Distance: inf
Reliability: 0.9025
Explored States: 3
Planning Time: 0.000141 seconds

Goal changed to G2

=====

PLANNING RESULT

=====

Success: True
State Path: S -> B -> G2
Total Cost: 3.2
Minimum Safety Distance: inf
Reliability: 0.9025
Explored States: 4
Planning Time: 8.9e-05 seconds

TEST CASE 6
Transition Addition

Before shortcut:

=====

PLANNING RESULT

=====

Success: True
State Path: S -> A -> G
Total Cost: 4.2
Minimum Safety Distance: inf
Reliability: 0.9025
Explored States: 3
Planning Time: 0.000137 seconds

Adding new shortcut S -> G

After shortcut:

CONCLUSION

The Safe Semantic Planner successfully finds paths from an initial state to a goal while avoiding bad states. The planner considers heuristic distance, safety and transition properties.

The six test cases demonstrate basic planning, safety-aware planning and dynamic replanning. The results show that the planner can adapt to changes in the environment and generate updated solutions.